

Midterm Code Report: NYT Surveillance Corpus Analysis

SIS/ITEC 724 — Spring 2026 — Professor Cogburn

Daniel Chavez

2026-03-04

Table of contents

1	CRISP-DM Step 1: Business/Research Understanding	7
2	Project Overview	7
3	Longitudinal analysis of NYT surveillance coverage (1995–2025),	7
4	comparing U.S. and PRC government surveillance framing across ~13,000 articles.	7
5	Data exported from ProQuest TDM Studio as two CSV files.	7
6	Disaggregated Research Questions (per Assignment 6 feedback):	7
7		7
8	RQ1: Longitudinal sentiment trends	7
9	RQ1.1: What is the overall trajectory of sentiment toward government	7
10	surveillance in NYT coverage from 1995–2025? (Bing lexicon, loess trend)	7
11	RQ1.2: Do VADER scores converge with dictionary-based results,	7
12	validating the trend? (VADER triangulation)	7
13		7
14	RQ2: Dominant frames and themes	7
15	RQ2.1: What are the most frequent terms and bigrams characterizing	7
16	surveillance coverage? (word freq, n-grams, TF-IDF — inductive/MVP core)	7
17	RQ2.2: What latent topics emerge from LDA modeling, and how do topic	7
18	proportions shift across eras? (LDA — elective extension)	7
19	RQ2.3: What emotional frames (NRC emotions) dominate coverage? (NRC lexicon)	7
20		7
21	RQ3: U.S. vs. PRC framing differences	7

22 RQ3.1: Which words statistically distinguish U.S. from PRC coverage?	7
23 (quanteda keyness/chi-squared — elective extension)	7
24 RQ3.2: How do sentiment trajectories differ between the two corpora?	7
25 (comparative sentiment — MVP core deductive)	7
26 RQ3.3: Does the semantic neighborhood of “surveillance” differ across	7
27 corpora? (GloVe + Word2Vec embeddings — elective extension)	7
28	7
29 RQ4: Event-driven shifts	7
30 RQ4.1: How did publication volume and sentiment shift around 9/11 (2001)	7
31 and the Snowden revelations (2013)? (event-window analysis)	7
32 RQ4.2: How did the emergence of AI-enabled surveillance (~2017) correspond	7
33 with changes in vocabulary and tone? (keyword tracking, embeddings)	7
34 CRISP-DM Step 2: Data Understanding & Collection	7
35 Part 1: Environment Setup	7
36 —————	7
37 Loads R packages across three text mining ecosystems (tm, tidytext,	7
38 quanteda) plus visualization, embeddings, and Python bridge.	7
38.1 Deliverable 1: Install R Packages	7
38.2 Deliverable 2: Load R Libraries	8
38.3 Deliverable 3: Verify Python Environment	8
39 Part 2: Data Import	9
40 —————	9
41 Data exported from ProQuest TDM Studio as two CSV files stored	9
42 in the Data/ subdirectory. Each row = one article with full text	9
43 and metadata. Combined into a single dataframe with corpus tag	9
44 and four analytical eras defined in the conceptual framework.	9
45 Corpus: ~7,621 U.S. articles + ~5,391 PRC articles = ~13,012 total.	9
45.1 Deliverable 4: Load CSV Data	9
46 Reads both CSVs, tags corpus of origin, combines, and creates	9
47 year and era variables. Column names may need adjustment after	9
48 inspecting actual CSV headers — check with head() or colnames().	9

49 CRISP-DM Step 3: Data Preparation	10
50 Part 3: Preprocessing	10
51 _____	10
52 Tokenization, stopwords removal, custom surveillance-specific	10
53 stopwords dictionary. All downstream analyses depend on this step.	10
53.1 Deliverable 5: Tokenize and Remove Stopwords	10
54 Breaks articles into unigrams. Removes standard English stopwords	10
55 plus custom list of high-frequency NYT noise terms.	10
56 Filters to alphabetic tokens, 2+ characters.	10
57 Supports: all RQs.	10
58 CRISP-DM Step 4: Modeling	11
59 Part 4: Inductive Analysis — MVP Core (RQ2.1)	11
60 _____	11
60.1 Deliverable 6: Word Frequencies	11
61 Answers RQ2.1 — dominant vocabulary per corpus.	11
62 Technique: unigram frequency counts grouped by corpus.	11
63 Viz: horizontal bar chart, faceted US vs. PRC, top 15 terms.	11
63.1 Deliverable 7: Bigram Analysis	12
64 Answers RQ2.1 — multi-word frames (e.g., “facial recognition,”	12
65 “social credit,” “national security”).	12
66 Technique: bigram tokenization, stopwords filter on both positions.	12
67 Viz: faceted bar chart, top 10 bigrams per corpus.	12
67.1 Deliverable 8: TF-IDF Analysis	13
68 Answers RQ2.1, bridges to RQ3.1 — distinctively important words	13
69 per corpus (not just frequent). Down-weights common terms.	13
70 Technique: <code>bind_tf_idf()</code> on corpus-level word counts.	13
71 Viz: faceted bar chart, top 10 TF-IDF terms per corpus.	13
72 Part 5: Deductive Analysis — MVP Core (RQ1, RQ3.2)	14
73 _____	14
73.1 Deliverable 9: Bing Lexicon Sentiment Scoring	14
74 Answers RQ1.1 and RQ3.2 — binary pos/neg via Bing lexicon.	14
75 Computes article-level net sentiment and ratio.	14

76 Wrangling: inner_join with Bing, pivot wide, compute scores.	14
77 Note: Bing is binary; no intensity or negation handling.	14
77.1 Deliverable 10: Longitudinal Sentiment Trend (RQ1.1)	15
78 Mean net sentiment per year, smoothed via loess.	15
79 Viz: scatter + loess by corpus, vertical lines at 9/11, Snowden, AI era.	15
80 Point size = article count; sparse years have wider CIs.	15
80.1 Deliverable 11: Sentiment by Era (RQ3.2, RQ4.1)	16
81 Compares mean sentiment between US and PRC across four eras.	16
82 Viz: grouped bar chart with standard error bars.	16
82.1 Deliverable 12: VADER Sentiment in Python (RQ1.2)	17
83 Triangulates Bing with VADER compound scores.	17
84 VADER handles negation and intensifiers better than unigram lexicons.	17
85 Convergence between Bing and VADER strengthens RQ1 findings.	17
86 Part 6: Elective Extensions	19
86.1 Deliverable 13–14: Quanteda Keyness (RQ3.1)	19
87 Chi-squared keyness identifies words used significantly more in	19
88 one corpus than the other. Accounts for corpus size imbalance	19
89 (7,621 vs 5,391) via expected vs. observed frequencies.	19
90 Viz: textplot_keyness, top 15 terms, blue=US, red=PRC.	19
90.1 Deliverable 15–16: GloVe Embeddings in R (RQ3.3, RQ4.2)	20
91 Captures semantic relationships beyond frequency. Queries nearest	20
92 neighbors for “surveillance” to see concept clustering.	20
93 Technique: text2vec TCM + GlobalVectors, rank=50, window=5.	20
94 With full data: train per-corpus and per-era models to measure	20
95 semantic drift (RQ4.2).	20
95.1 Deliverable 17: Word2Vec in Python (RQ3.3)	21
96 Second embedding method for triangulation with GloVe.	21
97 Technique: gensim Word2Vec skip-gram, dim=50, window=5.	21
98 With full data: train per-corpus models, compare neighbors.	21
99 CRISP-DM Step 5: Evaluation	22
100Part 7: Publication Volume & Emotion Analysis (RQ4, RQ2.3)	22
100.1Deliverable 18: Article Volume Over Time (RQ4.1, RQ4.2)	22
101Tracks coverage volume as proxy for editorial attention.	22

102	Volume spikes around events = initial evidence for RQ4.	22
103	Viz: stacked area chart with event marker lines.	22
	103.1 Deliverable 19: NRC Emotion Analysis (RQ2.3)	23
104	NRC lexicon classifies words into 8 emotions. Reveals which	23
105	emotional frames dominate each corpus.	23
106	Viz: grouped horizontal bar chart, US vs. PRC.	23
107	Note: NRC license must be accepted interactively on first run.	23
108	CRISP-DM Step 6: Deployment (Next Steps)	25
109	Conclusion and Next Steps	25
110		25
111	MVP Core (Completed):	25
112	Word frequency (Deliverable 6) → RQ2.1	25
113	Bigrams (Deliverable 7) → RQ2.1	25
114	TF-IDF (Deliverable 8) → RQ2.1, RQ3.1	25
115	Bing sentiment (Deliverables 9–11) → RQ1.1, RQ3.2, RQ4.1	25
116	Data visualizations throughout	25
117		25
118	Elective Extensions (In Progress):	25
119	VADER sentiment (Deliverable 12) → RQ1.2	25
120	Quanteda keyness (Deliverables 13–14) → RQ3.1	25
121	GloVe embeddings (Deliverables 15–16) → RQ3.3, RQ4.2	25
122	Word2Vec embeddings (Deliverable 17) → RQ3.3	25
123	NRC emotion profiling (Deliverable 19) → RQ2.3	25
124		25
125	Remaining for Final:	25
126	1. Add LDA topic modeling for RQ2.2 (test k = 5, 10, 15, 20)	25
127	2. Implement event-window analysis for RQ4.1 (12-month before/after)	25
128	3. Test multiple AI-era cutoffs (2015, 2017, 2019) for RQ4.2	25
129	4. Normalize comparative analyses for corpus size imbalance	25
130	5. Build GitHub repo for reproducibility	25

- 1 CRISP-DM Step 1: Business/Research Understanding
- 2 Project Overview
- 3 Longitudinal analysis of NYT surveillance coverage (1995–2025),
- 4 comparing U.S. and PRC government surveillance framing across
- ~13,000 articles.
- 5 Data exported from ProQuest TDM Studio as two CSV files.
- 6 Disaggregated Research Questions (per Assignment 6 feedback):
- 7
- 8 RQ1: Longitudinal sentiment trends
- 9 RQ1.1: What is the overall trajectory of sentiment toward gov-
- ernment
- 10 surveillance in NYT coverage from 1995–2025? (Bing lexicon,
- loess trend)
- 11 RQ1.2: Do VADER scores converge with dictionary-based
- results,
- 12 validating the trend? (VADER triangulation)
- 13
- 14 RQ2: Dominant frames and themes
- 15 RQ2.1: What are the most frequent terms and bigrams char-
- acterizing
- 16 surveillance coverage? (word freq, n-grams, TF-IDF — induc-
- tive/MVP core)
- 17 RQ2.2: What latent topics emerge from LDA modeling, and
- how do topic
- 18 proportions shift across eras? (LDA — elective extension)
- 19 RQ2.3: What emotional frames (NRC emotions) dominate
- coverage? (NRC lexicon)
- 20
- 21 RQ3: U.S. vs. PRC framing differences

```
install.packages(c("tidytext", "tm", "quanteda", "quanteda.textplots",
                  "quanteda.textstats", "tidyverse", "scales",
                  "textdata", "topicmodels", "text2vec"))
```

38.2 Deliverable 2: Load R Libraries

```
library(tidyverse)
library(tidytext)
library(tm)
library(quanteda)
library(quanteda.textplots)
library(quanteda.textstats)
library(scales)
```

38.3 Deliverable 3: Verify Python Environment

```
reticulate::py_config()
```

```
python:          /Users/chavez/.virtualenvs/r-reticulate/bin/python
libpython:       /Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/libpython3.9.dylib
pythonhome:      /Users/chavez/.virtualenvs/r-reticulate:/Users/chavez/.virtualenvs/r-reticulate
version:         3.9.6 (default, Dec 2 2025, 07:27:58) [Clang 17.0.0 (clang-1700.6.3.2)]
numpy:           /Users/chavez/.virtualenvs/r-reticulate/lib/python3.9/site-packages/numpy
numpy_version:   2.0.2
```

NOTE: Python version was forced by VIRTUAL_ENV

```
import numpy as np
import warnings
warnings.filterwarnings('ignore')

import nltk
nltk.download("stopwords", quiet=True)
```

True

```
nltk.download("punkt", quiet=True)
```

True

```
nltk.download("punkt_tab", quiet=True)
```

True

```
nltk.download("vader_lexicon", quiet=True)
```

True

```
from nltk.corpus import stopwords
from nltk.sentiment.vader import SentimentIntensityAnalyzer

print("Python environment verified successfully.")
```

Python environment verified successfully.

```
print(f"NumPy version: {np.__version__}")
```

NumPy version: 2.0.2

39 Part 2: Data Import

40

41 Data exported from ProQuest TDM Studio as two CSV files
stored

42 in the Data/ subdirectory. Each row = one article with full
text

43 and metadata. Combined into a single dataframe with corpus
tag

44 and four analytical eras defined in the conceptual framework.

45 Corpus: ~7,621 U.S. articles + ~5,391 PRC articles = ~13,012
total.

45.1 Deliverable 4: Load CSV Data

46 Reads both CSVs, tags corpus of origin, combines, and creates
47 year and era variables. Column names may need adjustment
after

48 inspecting actual CSV headers — check with `head()` or `col-
names()`.

```
us_raw <- read_csv("Data/NYT US Sentiment.csv", show_col_types = FALSE)
prc_raw <- read_csv("Data/NYT China Sentiment extended.csv", show_col_types = FALSE)

# Tag corpus of origin
us_raw$corpus <- "US"
prc_raw$corpus <- "PRC"

# Combine and build text field from Title + Subject Terms
# Date column is already parsed as date type by read_csv
sample_corpus <- bind_rows(us_raw, prc_raw) %>%
  mutate(
    doc_id = paste0(corpus, "_", GOID),
    pub_date = Date,
    year = year(pub_date),
    # Combine Title + Subject Terms into a single text field
```

```

# Subject Terms arrive as "['term1', 'term2']" - clean the brackets/quotes
clean_subjects = str_remove_all(`Subject Terms`, "\\[|\\]|'"),
fulltext = paste>Title, clean_subjects, sep = " "),
fulltext = str_squish(fulltext),
era = case_when(
  year < 2001 ~ "Pre-9/11",
  year >= 2001 & year < 2013 ~ "Post-9/11",
  year >= 2013 & year < 2017 ~ "Post-Snowden",
  year >= 2017 ~ "AI Era"
),
era = factor(era, levels = c("Pre-9/11", "Post-9/11", "Post-Snowden", "AI Era"))
) %>%
filter(!is.na(fulltext), fulltext != "")

cat(sprintf("Combined corpus: %d documents (%d US, %d PRC)\n",
  nrow(sample_corpus),
  sum(sample_corpus$corpus == "US"),
  sum(sample_corpus$corpus == "PRC")))

```

Combined corpus: 13012 documents (7621 US, 5391 PRC)

```
cat(sprintf("Year range: %d - %d\n", min(sample_corpus$year), max(sample_corpus$year)))
```

Year range: 1995 - 2025

49 CRISP-DM Step 3: Data Preparation

50 Part 3: Preprocessing

51

52 Tokenization, stopwords removal, custom surveillance-specific

53 stopwords dictionary. All downstream analyses depend on this step.

53.1 Deliverable 5: Tokenize and Remove Stopwords

54 Breaks articles into unigrams. Removes standard English stopwords

55 plus custom list of high-frequency NYT noise terms.

56 Filters to alphabetic tokens, 2+ characters.

57 Supports: all RQs.

```

custom_stops <- tibble(word = c("said", "new", "york", "times", "mr", "ms",
  "also", "would", "could", "one", "two",
  "like", "just", "even", "say", "says",

```

```

                                "article", "percent", "year", "years"))

tokens <- sample_corpus %>%
  unnest_tokens(word, fulltext) %>%
  anti_join(stop_words, by = "word") %>%
  anti_join(custom_stops, by = "word") %>%
  filter(str_detect(word, "^[a-z]{2,}$"))

cat(sprintf("Total tokens after preprocessing: %s\n", comma(nrow(tokens))))

```

Total tokens after preprocessing: 176,672

```
cat(sprintf("Unique vocabulary: %s terms\n", comma(n_distinct(tokens$word))))
```

Unique vocabulary: 12,374 terms

58 CRISP-DM Step 4: Modeling

59 Part 4: Inductive Analysis — MVP Core (RQ2.1)

60

60.1 Deliverable 6: Word Frequencies

61 Answers RQ2.1 — dominant vocabulary per corpus.

62 Technique: unigram frequency counts grouped by corpus.

63 Viz: horizontal bar chart, faceted US vs. PRC, top 15 terms.

```

word_counts <- tokens %>%
  count(corpus, word, sort = TRUE)

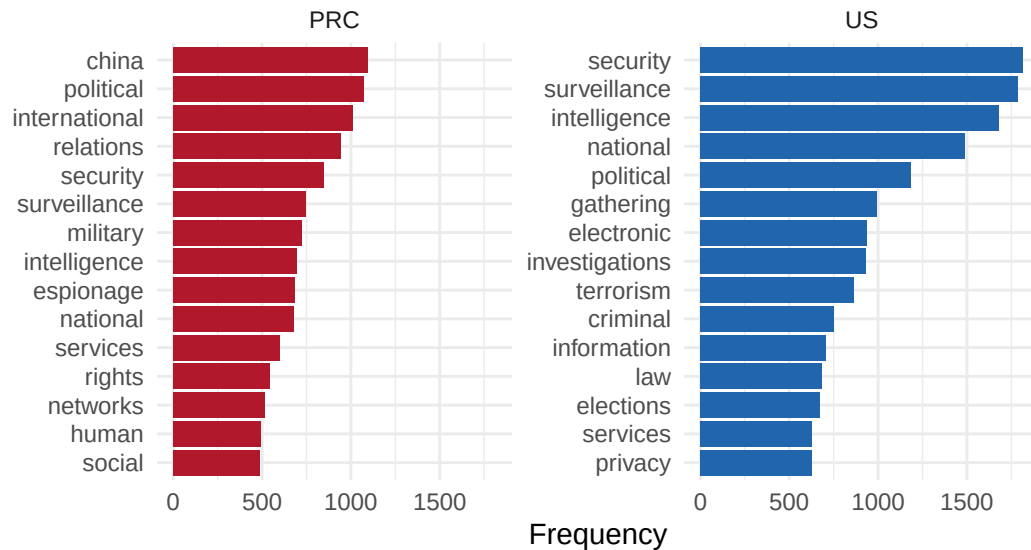
top_words <- word_counts %>%
  group_by(corpus) %>%
  slice_max(n, n = 15) %>%
  ungroup() %>%
  mutate(word = reorder_within(word, n, corpus))

ggplot(top_words, aes(x = n, y = word, fill = corpus)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ corpus, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Top 15 Most Frequent Words by Corpus",
       subtitle = "After stopword removal | NYT Surveillance Corpus",
       x = "Frequency", y = NULL) +
  theme_minimal(base_size = 11)

```

Top 15 Most Frequent Words by Corpus

After stopwords removal | NYT Surveillance Corpus



63.1 Deliverable 7: Bigram Analysis

64 Answers RQ2.1 — multi-word frames (e.g., “facial recognition,”

65 “social credit,” “national security”).

66 Technique: bigram tokenization, stopwords filter on both positions.

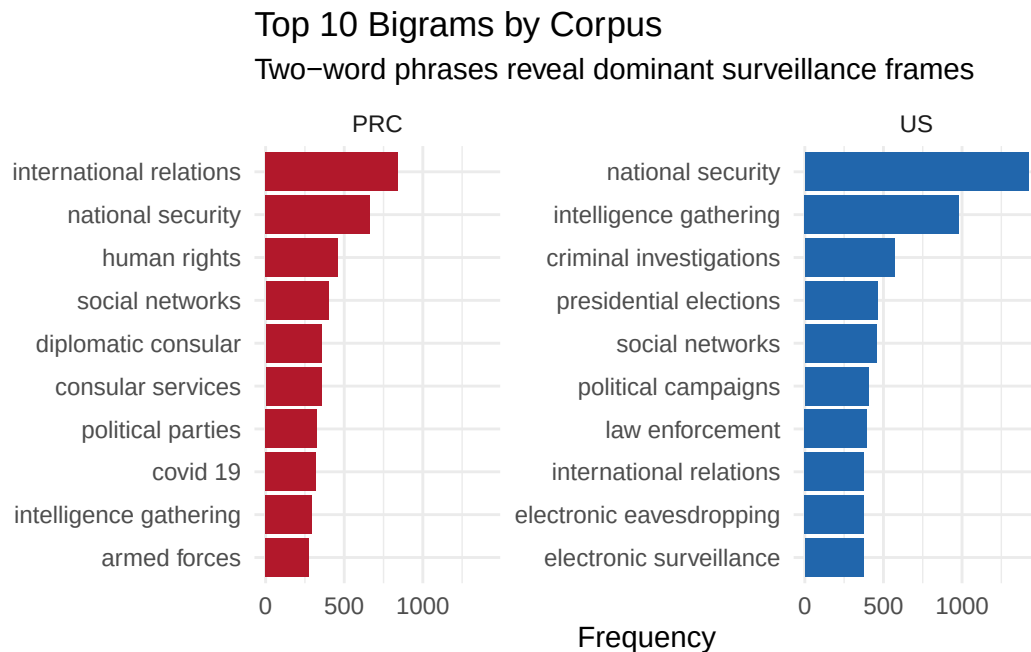
67 Viz: faceted bar chart, top 10 bigrams per corpus.

```
bigrams <- sample_corpus %>%
  unnest_tokens(bigram, fulltext, token = "ngrams", n = 2) %>%
  separate(bigram, into = c("word1", "word2"), sep = " ") %>%
  filter(!word1 %in% stop_words$word,
         !word2 %in% stop_words$word,
         !word1 %in% custom_stops$word,
         !word2 %in% custom_stops$word) %>%
  unite(bigram, word1, word2, sep = " ")

top_bigrams <- bigrams %>%
  count(corpus, bigram, sort = TRUE) %>%
  group_by(corpus) %>%
  slice_max(n, n = 10) %>%
  ungroup() %>%
  mutate(bigram = reorder_within(bigram, n, corpus))

ggplot(top_bigrams, aes(x = n, y = bigram, fill = corpus)) +
  geom_col(show.legend = FALSE) +
```

```
facet_wrap(~ corpus, scales = "free_y") +
scale_y_reordered() +
scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
labs(title = "Top 10 Bigrams by Corpus",
      subtitle = "Two-word phrases reveal dominant surveillance frames",
      x = "Frequency", y = NULL) +
theme_minimal(base_size = 11)
```



67.1 Deliverable 8: TF-IDF Analysis

68 Answers RQ2.1, bridges to RQ3.1 — distinctively important words

69 per corpus (not just frequent). Down-weights common terms.

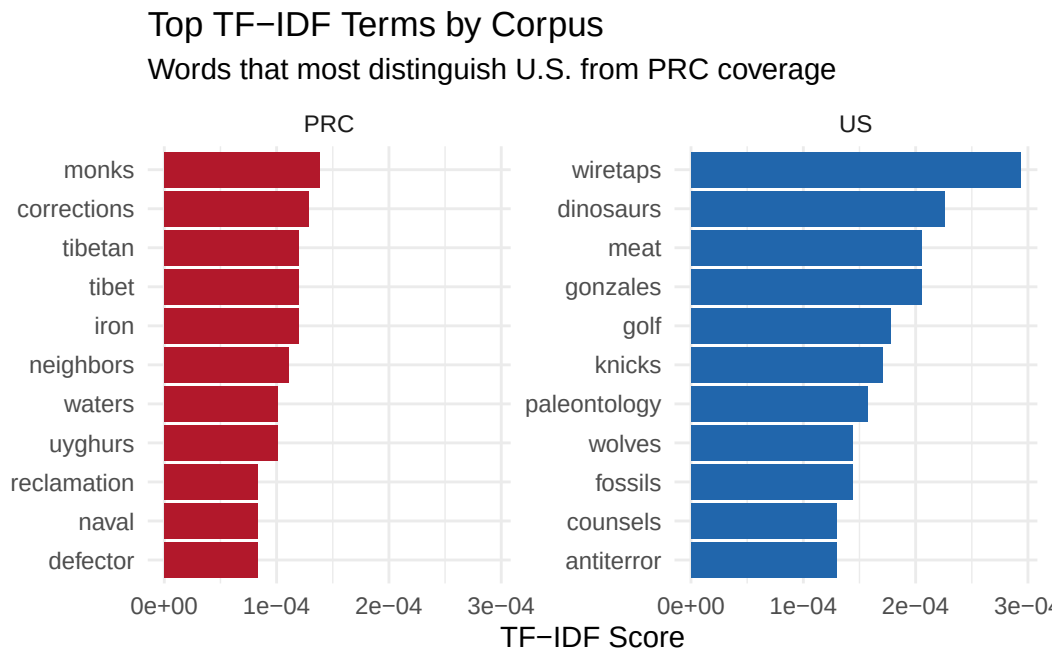
70 Technique: `bind_tf_idf()` on corpus-level word counts.

71 Viz: faceted bar chart, top 10 TF-IDF terms per corpus.

```
corpus_tf_idf <- tokens %>%
  count(corpus, word) %>%
  bind_tf_idf(word, corpus, n) %>%
  arrange(desc(tf_idf))

top_tf_idf <- corpus_tf_idf %>%
  group_by(corpus) %>%
  slice_max(tf_idf, n = 10) %>%
  ungroup() %>%
  mutate(word = reorder_within(word, tf_idf, corpus))
```

```
ggplot(top_tfidf, aes(x = tf_idf, y = word, fill = corpus)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ corpus, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Top TF-IDF Terms by Corpus",
       subtitle = "Words that most distinguish U.S. from PRC coverage",
       x = "TF-IDF Score", y = NULL) +
  theme_minimal(base_size = 11)
```



72 Part 5: Deductive Analysis — MVP Core (RQ1, RQ3.2)

73

73.1 Deliverable 9: Bing Lexicon Sentiment Scoring

74 Answers RQ1.1 and RQ3.2 — binary pos/neg via Bing lexicon.

75 Computes article-level net sentiment and ratio.

76 Wrangling: inner_join with Bing, pivot wide, compute scores.

77 Note: Bing is binary; no intensity or negation handling.

```
bing <- get_sentiments("bing")

sentiment_by_article <- tokens %>%
  inner_join(bing, by = "word") %>%
  count(doc_id, corpus, year, era, sentiment) %>%
```

```

pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
mutate(net_sentiment = positive - negative,
       total_words = positive + negative,
       sentiment_ratio = net_sentiment / total_words)

cat(sprintf("Articles with sentiment data: %d of %d\n",
           n_distinct(sentiment_by_article$doc_id),
           nrow(sample_corpus)))

```

Articles with sentiment data: 8974 of 13012

77.1 Deliverable 10: Longitudinal Sentiment Trend (RQ1.1)

78 Mean net sentiment per year, smoothed via loess.

79 Viz: scatter + loess by corpus, vertical lines at 9/11, Snowden, AI era.

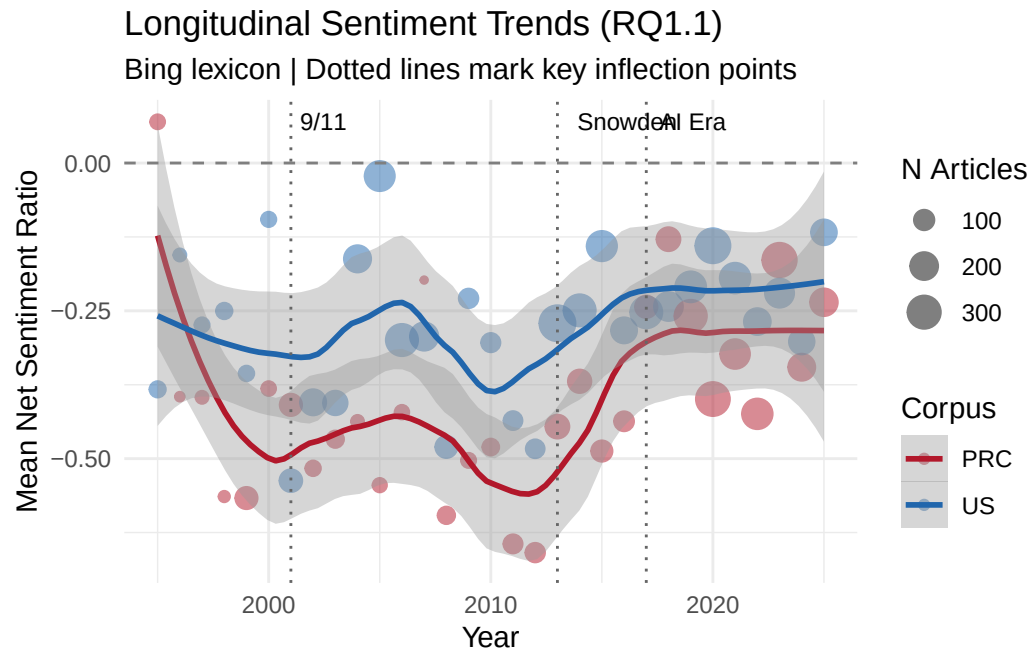
80 Point size = article count; sparse years have wider CIs.

```

yearly_sentiment <- sentiment_by_article %>%
  group_by(year, corpus) %>%
  summarize(
    mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
    n_articles = n(),
    .groups = "drop"
  )

ggplot(yearly_sentiment, aes(x = year, y = mean_sentiment, color = corpus)) +
  geom_point(aes(size = n_articles), alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, span = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray50") +
  geom_vline(xintercept = c(2001, 2013, 2017), linetype = "dotted",
            color = "gray40", linewidth = 0.5) +
  ggplot2::annotate("text", x = 2001, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                    label = "9/11", hjust = -0.2, size = 3) +
  ggplot2::annotate("text", x = 2013, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                    label = "Snowden", hjust = -0.2, size = 3) +
  ggplot2::annotate("text", x = 2017, y = max(yearly_sentiment$mean_sentiment, na.rm = TRUE),
                    label = "AI Era", hjust = -0.2, size = 3) +
  scale_color_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Longitudinal Sentiment Trends (RQ1.1)",
       subtitle = "Bing lexicon | Dotted lines mark key inflection points",
       x = "Year", y = "Mean Net Sentiment Ratio",
       color = "Corpus", size = "N Articles") +
  theme_minimal(base_size = 11)

```



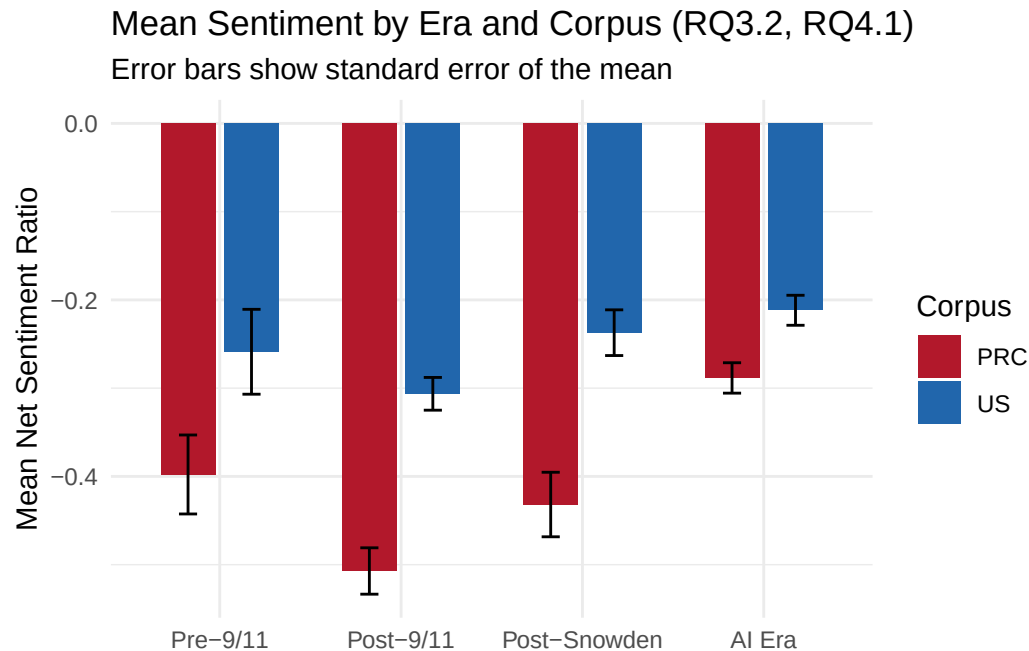
80.1 Deliverable 11: Sentiment by Era (RQ3.2, RQ4.1)

81 Compares mean sentiment between US and PRC across four eras.

82 Viz: grouped bar chart with standard error bars.

```
era_sentiment <- sentiment_by_article %>%
  group_by(era, corpus) %>%
  summarize(
    mean_sentiment = mean(sentiment_ratio, na.rm = TRUE),
    sd_sentiment = sd(sentiment_ratio, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )

ggplot(era_sentiment, aes(x = era, y = mean_sentiment, fill = corpus)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_errorbar(
    aes(ymin = mean_sentiment - sd_sentiment / sqrt(n),
        ymax = mean_sentiment + sd_sentiment / sqrt(n)),
    position = position_dodge(width = 0.7), width = 0.2
  ) +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "Mean Sentiment by Era and Corpus (RQ3.2, RQ4.1)",
       subtitle = "Error bars show standard error of the mean",
       x = NULL, y = "Mean Net Sentiment Ratio", fill = "Corpus") +
  theme_minimal(base_size = 11)
```



82.1 Deliverable 12: VADER Sentiment in Python (RQ1.2)

83 Triangulates Bing with VADER compound scores.

84 VADER handles negation and intensifiers better than unigram lexicons.

85 Convergence between Bing and VADER strengthens RQ1 findings.

```
import pandas as pd
from nltk.sentiment.vader import SentimentIntensityAnalyzer

sid = SentimentIntensityAnalyzer()
corpus_data = pd.DataFrame(r.sample_corpus)

vader_scores = []
for idx, row in corpus_data.iterrows():
    scores = sid.polarity_scores(str(row['fulltext']))
    vader_scores.append({
        'doc_id': row['doc_id'],
        'corpus': row['corpus'],
        'year': int(row['year']),
        'compound': scores['compound'],
        'pos': scores['pos'],
        'neg': scores['neg'],
        'neu': scores['neu']
    })
```

```
print(f"VADER scored {len(vader_scores)} documents.")
```

VADER scored 13012 documents.

```
print(f"Mean compound (US): {sum(s['compound'] for s in vader_scores if s['corpus']=='US') / max(1, sum(s['compound'] for s in vader_scores if s['corpus']=='US'))}")
```

Mean compound (US): -0.043

```
print(f"Mean compound (PRC): {sum(s['compound'] for s in vader_scores if s['corpus']=='PRC') / max(1, sum(s['compound'] for s in vader_scores if s['corpus']=='PRC'))}")
```

Mean compound (PRC): -0.106

```
vader_df <- py$vader_scores %>%
```

```
  bind_rows() %>%
```

```
  mutate(year = as.integer(year))
```

```
vader_yearly <- vader_df %>%
```

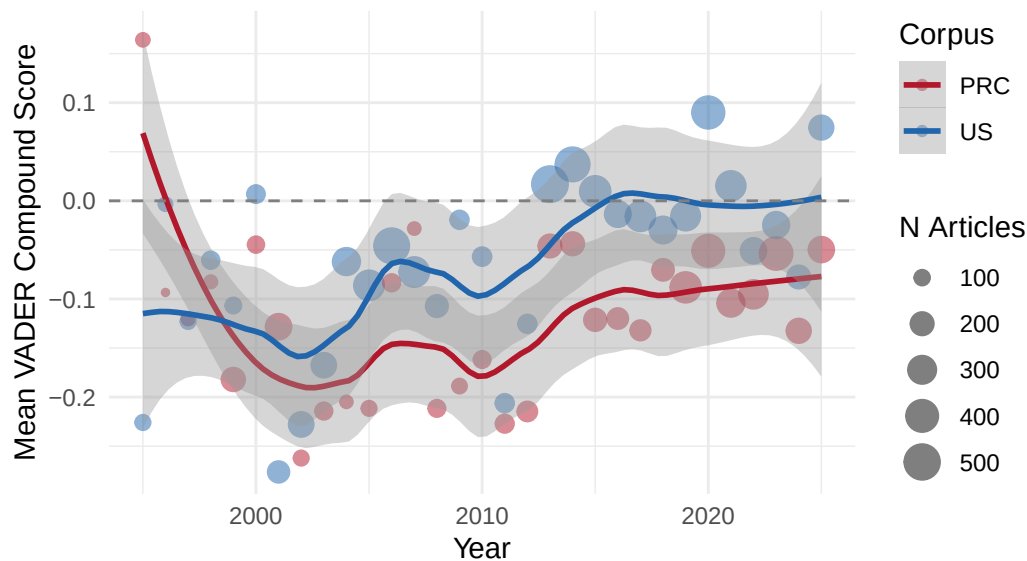
```
  group_by(year, corpus) %>%
```

```
  summarize(mean_compound = mean(compound, na.rm = TRUE),
            n = n(), .groups = "drop")
```

```
ggplot(vader_yearly, aes(x = year, y = mean_compound, color = corpus)) +
  geom_point(aes(size = n), alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, span = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray50") +
  scale_color_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "VADER Compound Sentiment Over Time (RQ1.2)",
       subtitle = "Python VADER via reticulate | Triangulation with Bing",
       x = "Year", y = "Mean VADER Compound Score",
       color = "Corpus", size = "N Articles") +
  theme_minimal(base_size = 11)
```

VADER Compound Sentiment Over Time (RQ1.2)

Python VADER via reticulate | Triangulation with Bing



86 Part 6: Elective Extensions

86.1 Deliverable 13–14: Quanteda Keynes (RQ3.1)

87 Chi-squared keyness identifies words used significantly more in
88 one corpus than the other. Accounts for corpus size imbalance
89 (7,621 vs 5,391) via expected vs. observed frequencies.
90 Viz: `textplot_keyness`, top 15 terms, blue=US, red=PRC.

```
library(text2vec)

qcorp <- corpus(sample_corpus, text_field = "fulltext",
                docid_field = "doc_id")

qtoks <- tokens(qcorp, remove_punct = TRUE, remove_numbers = TRUE) %>%
  tokens_tolower() %>%
  tokens_remove(c(stopwords("en"), custom_stops$word))

qdfm <- dfm(qtoks)

cat(sprintf("DFM dimensions: %d documents × %d features\n",
            nrow(qdfm), ncol(qdfm)))
```

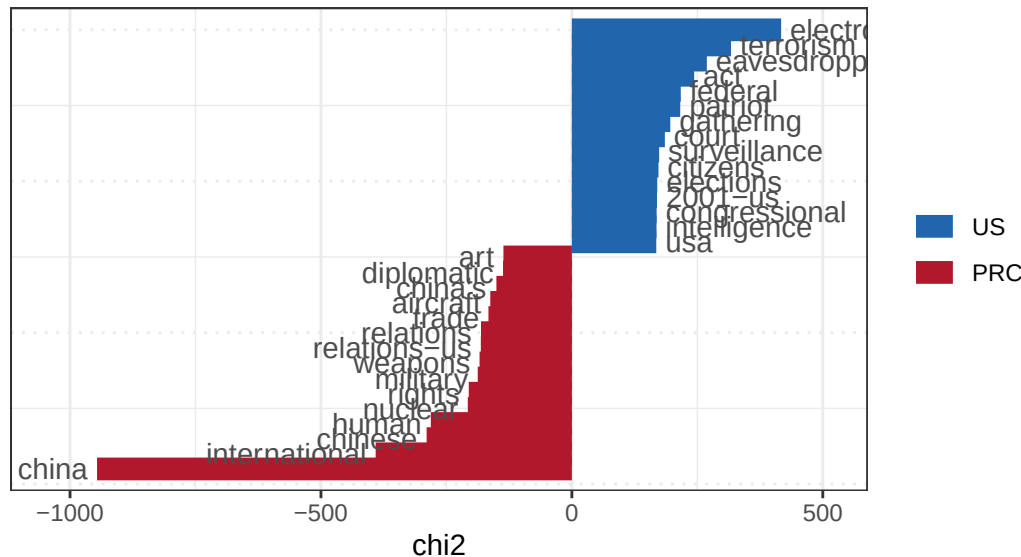
DFM dimensions: 13012 documents × 14004 features

```
dfm_grouped <- dfm_group(qdfm, groups = corpus)
keyness_results <- textstat_keyness(dfm_grouped, target = "US")

textplot_keyness(keyness_results, n = 15,
                  color = c("#2166AC", "#B2182B")) +
  labs(title = "Keyness: U.S. vs. PRC Surveillance Vocabulary (RQ3.1)",
        subtitle = "Chi-squared test | Blue = U.S. | Red = PRC")
```

Keyness: U.S. vs. PRC Surveillance Vocabulary (RQ3.1)

Chi-squared test | Blue = U.S. | Red = PRC



90.1 Deliverable 15–16: GloVe Embeddings in R (RQ3.3, RQ4.2)

- 91 Captures semantic relationships beyond frequency. Queries nearest
- 92 neighbors for “surveillance” to see concept clustering.
- 93 Technique: text2vec TCM + GlobalVectors, rank=50, window=5.
- 94 With full data: train per-corpus and per-era models to measure
- 95 semantic drift (RQ4.2).

```
t2v_tokens <- sample_corpus$fulltext %>%
  tolower() %>%
  word_tokenizer()

it <- itoken(t2v_tokens, progressbar = FALSE)
vocab <- create_vocabulary(it) %>%
  prune_vocabulary(term_count_min = 3)
vectorizer <- vocab_vectorizer(vocab)
tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)

glove_model <- GlobalVectors$new(rank = 50, x_max = 10)
word_vectors <- glove_model$fit_transform(tcm, n_iter = 20,
  convergence_tol = 0.01)

cat(sprintf("GloVe embeddings: %d words × %d dimensions\n",
```

```
nrow(word_vectors), ncol(word_vectors)))
```

GloVe embeddings: 6287 words × 50 dimensions

```
if ("surveillance" %in% rownames(word_vectors)) {
  target_vec <- word_vectors["surveillance", , drop = FALSE]
  cos_sims <- sim2(x = word_vectors, y = target_vec,
                  method = "cosine", norm = "l2")
  nearest <- sort(drop(cos_sims[,1]), decreasing = TRUE)

  cat("Top 10 words most similar to 'surveillance':\n")
  print(head(names(nearest)[names(nearest) != "surveillance"], 10))
} else {
  cat("'surveillance' not in vocabulary (try with full corpus).\n")
}
```

Top 10 words most similar to 'surveillance':

```
[1] "gathering"      "intelligence"  "investigations" "electronic"
[5] "privacy"        "citizens"      "security"        "networks"
[9] "national"       "law"
```

95.1 Deliverable 17: Word2Vec in Python (RQ3.3)

96 Second embedding method for triangulation with GloVe.

97 Technique: gensim Word2Vec skip-gram, dim=50, window=5.

98 With full data: train per-corpus models, compare neighbors.

```
from gensim.models import Word2Vec
import math

corpus_data = r.sample_corpus

stop = set(__import__('nltk').corpus.stopwords.words("english"))
documents = []
for text in corpus_data['fulltext']:
    words = [w.lower() for w in str(text).split() if w.isalpha() and w.lower() not in stop]
    if words:
        documents.append(words)

model = Word2Vec(
    sentences=documents,
    vector_size=50,
    window=5,
    min_count=2,
    workers=1,
    sg=1,
    negative=10,
    epochs=10
)

kv = model.wv
```

```
print(f"Word2Vec vocab size: {len(kv)}")
```

Word2Vec vocab size: 6746

```
def cosine_py(a, b):
    num = sum(x*y for x, y in zip(a, b))
    den = math.sqrt(sum(x*x for x in a)) * math.sqrt(sum(y*y for y in b))
    return (num / den) if den else 0.0

if "surveillance" in kv.key_to_index:
    q = kv.get_vector("surveillance").tolist()
    sims = []
    for w in kv.key_to_index.keys():
        if w == "surveillance":
            continue
        v = kv.get_vector(w).tolist()
        sims.append((w, cosine_py(q, v)))
    sims.sort(key=lambda x: x[1], reverse=True)
    print("\nWords most similar to 'surveillance' (Word2Vec):")
    for word, score in sims[:10]:
        print(f" {word}: {score:.3f}")
```

Words most similar to 'surveillance' (Word2Vec):

```
citizens: 0.837
telephone: 0.811
warrantless: 0.782
legality: 0.776
violated: 0.770
fisa: 0.769
withier: 0.769
r: 0.760
trap: 0.759
alberto: 0.758
```

99 CRISP-DM Step 5: Evaluation

100 Part 7: Publication Volume & Emotion Analysis (RQ4, RQ2.3)

100.1 Deliverable 18: Article Volume Over Time (RQ4.1, RQ4.2)

101 Tracks coverage volume as proxy for editorial attention.

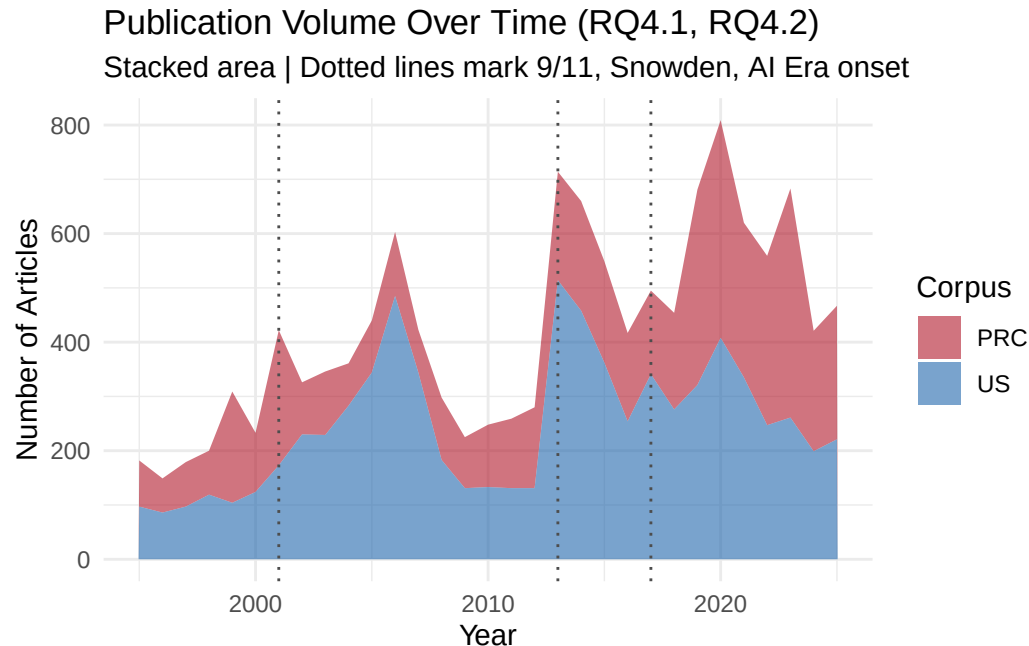
102 Volume spikes around events = initial evidence for RQ4.

103 Viz: stacked area chart with event marker lines.

```
volume <- sample_corpus %>%
  count(year, corpus)

ggplot(volume, aes(x = year, y = n, fill = corpus)) +
  geom_area(alpha = 0.6, position = "stack") +
```

```
geom_vline(xintercept = c(2001, 2013, 2017), linetype = "dotted",
           color = "gray30") +
scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
labs(title = "Publication Volume Over Time (RQ4.1, RQ4.2)",
     subtitle = "Stacked area | Dotted lines mark 9/11, Snowden, AI Era onset",
     x = "Year", y = "Number of Articles", fill = "Corpus") +
theme_minimal(base_size = 11)
```



103.1 Deliverable 19: NRC Emotion Analysis (RQ2.3)

104 NRC lexicon classifies words into 8 emotions. Reveals which

105 emotional frames dominate each corpus.

106 Viz: grouped horizontal bar chart, US vs. PRC.

107 Note: NRC license must be accepted interactively on first run.

```
# First run: accept NRC license interactively via get_sentiments("nrc")
# Then set eval: true

nrc <- get_sentiments("nrc") %>%
  filter(!sentiment %in% c("positive", "negative"))

emotion_counts <- tokens %>%
  inner_join(nrc, by = "word") %>%
  count(corpus, sentiment, sort = TRUE)
```

```

ggplot(emotion_counts, aes(x = reorder(sentiment, n), y = n, fill = corpus)) +
  geom_col(position = "dodge") +
  coord_flip() +
  scale_fill_manual(values = c("US" = "#2166AC", "PRC" = "#B2182B")) +
  labs(title = "NRC Emotion Profiles by Corpus (RQ2.3)",
       subtitle = "Which emotional frames dominate each corpus?",
       x = NULL, y = "Word Count", fill = "Corpus") +
  theme_minimal(base_size = 11)

```

108 CRISP-DM Step 6: Deployment (Next Steps)

109 Conclusion and Next Steps

110

111 MVP Core (Completed):

112 - Word frequency (Deliverable 6) → RQ2.1

113 - Bigrams (Deliverable 7) → RQ2.1

114 - TF-IDF (Deliverable 8) → RQ2.1, RQ3.1

115 - Bing sentiment (Deliverables 9–11) → RQ1.1, RQ3.2, RQ4.1

116 - Data visualizations throughout

117

118 Elective Extensions (In Progress):

119 - VADER sentiment (Deliverable 12) → RQ1.2

120 - Quantada keyness (Deliverables 13–14) → RQ3.1

121 - GloVe embeddings (Deliverables 15–16) → RQ3.3, RQ4.2

122 - Word2Vec embeddings (Deliverable 17) → RQ3.3

123 - NRC emotion profiling (Deliverable 19) → RQ2.3

124

125 Remaining for Final:

126 1. Add LDA topic modeling for RQ2.2 (test $k = 5, 10, 15, 20$)

127 2. Implement event-window analysis for RQ4.1 (12-month
before/after)

128 3. Test multiple AI-era cutoffs (2015, 2017, 2019) for RQ4.2

129 4. Normalize comparative analyses for corpus size imbalance

130 5. Build GitHub repo for reproducibility